

Attributes

Attributes are variables associated with an object. The Python documentation describes attributes as “any name following a dot.” For example, consider the `birthday.year` expression in the previous section. The `year` attribute is a name following a dot.

Every object has its own set of attributes. When the *wcexample1.py* program created two `WizCoin` objects and stored them in the `purse` and `coinJar` variables, their attributes had different values. You can access and set these attributes just like any variable. To practice setting attributes, open a new file editor window and enter the following code, saving it as *wcexample2.py* in the same folder as the *wizcoin.py* file:

```
import wizcoin

change = wizcoin.WizCoin(9, 7, 20)
print(change.sickles) # Prints 7.
change.sickles += 10
print(change.sickles) # Prints 17.

pile = wizcoin.WizCoin(2, 3, 31)
print(pile.sickles) # Prints 3.
pile.someNewAttribute = 'a new attr' # A new attribute is created.
print(pile.someNewAttribute)
```

When you run this program, the output looks like this:

```
7
17
3
a new attr
```

You can think of an object’s attributes as similar to a dictionary’s keys. You can read and modify their associated values and assign an object new attributes. Technically, methods are considered attributes of a class, as well.

Private Attributes and Private Methods

In languages such as C++ or Java, attributes can be marked as having *private access*, which means the compiler or interpreter only lets code